

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



REPORT
PROGRAMMING INTERGRATION PROJECT

KNOWLEDGE RETRIEVAL AND QA FOR
COMPUTER SCIENCE COURSES UTILIZING
RAG

MAJOR: COMPUTER SCIENCE

THESIS COMMITTEE : COMPUTER SCIENCE

SUPERVISOR(s) : NGUYỄN HỨA PHÙNG, Ph.D.

—————o0o—————

STUDENT : NGUYỄN QUỐC HIỆU - 2352344
: NGUYỄN HỮU CẦU - 2352129

HO CHI MINH CITY, May 2024

Contents

1	Preprocessing	1
1.1	Data	1
1.1.1	Collection	1
1.1.2	Data Handling	2
1.1.3	Text Converting using OCR	3
1.2	Syllabus Extraction	5
1.2.1	Goal	5
1.2.2	Inputs	5
1.2.3	Method	5
1.2.4	Patterns	6
1.2.5	Data Model & Metadata	6
1.2.6	Notes on Robustness	10
1.3	Material Extraction	11
1.3.1	Goal	11
1.3.2	Inputs	11
1.3.3	Method	11
1.3.4	Data Model & Metadata	12
1.4	Text Chunking	14
1.4.1	Scope and role	14
1.4.2	Inputs	14
1.4.3	Method	14
1.4.4	Chunk IDs, prefixes, and metadata	15
1.4.5	Complexity and expected chunk count	16

1.4.6	Example outputs	17
2	Vector Embedding	18
2.1	Concept	18
2.2	Embedding Model	19
2.3	Relationship with Chunking	20
2.4	Embedding Generation in the Pipeline	21
2.5	Embedding Indexing: FAISS IndexFlatIP	22
2.6	Summary	23
3	Indexing	24
3.1	Overview	24
3.2	Vector Indexing	25
3.2.1	Chunk Collection	25
3.2.2	Creating the FAISS Index	26
3.2.3	Adding Embeddings to Index	26
3.2.4	Metadata Store	26
3.2.5	Saving the Index	27
3.3	Searching	28
3.4	End-to-End Indexing Command	28
3.5	Summary	29
4	Add-ons	30

List of Figures

List of Tables

Table 1.1	Field glossary for syllabus.v1	8
-----------	--	---

Chapter 1

Preprocessing

This chapter describes how raw course materials were collected and transformed before retrieval and generation. It covers (i) data collection, (ii) data handling/normalization, and (iii) text conversion via OCR.

1.1 Data

1.1.1 Collection

Materials were gathered from [HCMUT LMS](#) and [Coursewave \(HCMUT alumni materials\)](#). The corpus includes the syllabus and lecture slides (PDF/Office). A representative subset of course folders:

```
CO1005_Introduction_to_Computing/  
CO1007_Discrete_Structures_for_Computer_Science/  
CO1023_Digital_Systems/  
CO1027_Programming_Fundamentals/  
CO2001_Professional_Skills_for_Engineers/  
CO2003_Data_Structures_and_Algorithms/  
CO2007_Computer_Architecture/  
CO2011_Mathematical_Modeling/  
CO2013_Database_Systems/  
CO2017_Operating_Systems/
```

```
CO2039_Advanced_Programming/
CO3001_Software_Engineering/
CO3005_Principles_of_Programming_Languages/
CO3023_Distributed_and_Object-Oriented_Systems/
CO3027_Electronic_Commerce/
CO3029_Data_Mining/
CO3031_Algorithms_Design_and_Analysis/
CO3033_Information_System_Security/
CO3037_Internet_of_Things_Applications/
CO3039_Computer_Networks/
CO3043_Mobile_Application_Development/
CO3045_Game_Programming/
CO3049_Web_Programming/
```

1.1.2 Data Handling

All files are normalized to a consistent directory layout and metadata schema. Each course directory contains raw sources, rendered page images, parsed JSON outputs, and vector indices:

```
data/
  <course>/
    raw/          # PDFs, DOCX, PPTX from source
    converted/    # page_001.png, page_002.png ... (220 DPI)
    data/
      syllabus/
        parsed/    # <course>.syllabus.json (normalized fields)
      material/
        material.json  # content units extracted per page
    indices/
      index.faiss    # FAISS IndexFlatIP (384-d, L2-normalized)
      metadata.json  # chunk_id -> {text, course_id, page_index, ...}
```

We maintain per-document metadata:

- **identifiers:** `course_id`, `course_name`, chapter title, `page_index`, and document type (syllabus/material);
- **provenance:** source path, file hash, and timestamp of ingestion;

1.1.3 Text Converting using OCR

PDFs are rasterized to page images at 220 DPI, then processed with **PaddleOCR 3.2** to obtain line-level text, a confidence score, and quadrilateral bounding polygons. We do *not* use LayoutLM or other layout models; any structure used later (e.g., headers/tables) is inferred only when needed from the OCR text and polygons.

Each page is serialized to JSON with a compact schema:

```
{
  "meta": {
    "source_path": ".../CO2039_Advanced_Programming/Syllabus/slide_001.png",
    "course": "Advanced_Programming",
    "engine": "paddleocr"
  },
  "items": [
    {
      "text": "HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY",
      "score": 0.987,
      "polygon": [[308,144],[923,144],[923,164],[308,164]]
    },
    {
      "text": "Course Syllabus",
      "score": 0.971,
      "polygon": [[724,513],[1106,517],[1106,562],[723,559]]
    }
  ]
}
```



```
// ... one entry per detected line  
]  
}
```

Region-constrained extraction. We use the OCR geometry (bbox/polygon) as spatial constraints to extract text from the per-line dictionary items returned by the detector (each item carries keys such as text, score, polygon). Only items whose geometry falls inside a region of interest are selected; their text values are ordered by reading sequence and concatenated. The resulting text is then normalized and passed to the downstream chunking and indexing stages.

1.2 Syllabus Extraction

1.2.1 Goal

From OCR line items (text + polygon), extract

- course info
- an assessment breakdown, robust to one- or two-table layouts

1.2.2 Inputs

Each OCR item is a dictionary with at least text, score, and polygon (four points). We first convert items into *records* containing centroid and bounding box coordinates for spatial matching. :contentReference[oaicite:0]index=0

```
{ "text": "...", "score": 0.98, "polygon": [[x1,y1],[x4,y4]] }  
---> record: { "text": "...", "x","y","x_min","x_max","y_min","y_max" }
```

1.2.3 Method

1. **Recordification.** Compute centroids and (x/y) min–max for all items to enable ordering and box tests.
2. **Header extraction (regex).** Pull course *title*, *course_id*, *credits*, and *applied semester* using tolerant patterns.
3. **Layout detection.** Use anchored keywords (e.g., “Projects”) count to decide between single-table vs dual-table layout.
4. **Column localization.** Locate column headers (Lessons/Hours, Credits, Ratio, Evaluation type/Format, Duration), then define padded X-ranges per column.
5. **Row extraction.** For each row anchor (Lectures, Tutorial, Labs, Projects, Self-study, Others), restrict to a Y-band around the row text and pick cell

values by intersecting with the column X-ranges. Spatial predicate supports center, overlap, or inside.

6. **Special case (Lectures).** When the syllabus lists two exam types (Midterm/Final) for the Lectures row, collect both with their ratios and durations as separate `EvaluationType` entries.
7. **Numeric parsing.** Convert hour/credit/ratio/duration strings robustly (commas, decimals) to integers.
8. **Object assembly.** Emit `CourseInfo` and a list of `AssessmentComponent` (with optional `EvaluationType`).

1.2.4 Patterns

We rely on anchored, case-insensitive patterns for headers, row names, and value pickers; examples: `title`, `course_id`, `credits`, `col_lessons`, `col_credits`, `col_ratio`, `col_evaluate`, `col_duration`, and numeric pickers like `ass_ratio` and `ass_duration`.

1.2.5 Data Model & Metadata

The syllabus extractor emits a JSON object with top-level keys `schema_version`, `metadata`, `course_info`, `assessments`, `course_des`, and `raw_text`. A representative example:

```
1 {  
2   "schema_version": "syllabus.v1",  
3   "metadata": {  
4     "doc_type": "syllabus",  
5     "course_id": "CO2039",  
6     "source_file": ".../CO2039_Advanced_Programming/Syllabus/slide_001.  
   png",  
7     "page_index": 0,
```

```

8   "language": "en",
9   "ocr_engine": "PaddleOCR 3.2",
10  "extractor_version": "1.0.0",
11  "timestamp": "2025-10-12T20:12:23+07:00"
12 },
13 "course_info": {
14   "title": "Advanced_Programming",
15   "course_id": "CO2039",
16   "credits": 0,
17   "applied_semester": "HK202"
18 },
19 "assessments": [
20   {
21     "name": "Lectures",
22     "hours": 0,
23     "credits": 0,
24     "ratio": null,
25     "evaluation_type": [
26       { "name": "Midterm exam", "ratio": 0, "duration_min": null },
27       { "name": "Final exam", "ratio": 60, "duration_min": 120 }
28     ]
29   },
30   { "name": "Tutorial", "hours": 0, "credits": 0, "ratio": 10, "
    evaluation_type": [] },
31   { "name": "Labs/Practices", "hours": 0, "credits": 0, "ratio": 0, "
    evaluation_type": [] },
32   { "name": "Projects", "hours": 0, "credits": 0, "ratio": 30, "
    evaluation_type": [] },
33   { "name": "Self-study", "hours": 0, "credits": 0, "ratio": 0, "
    evaluation_type": [] },
34   { "name": "Others", "hours": 0, "credits": 0, "ratio": 0, "
    evaluation_type": [] }

```

```

35 ],
36 "course_des": {
37     "description": null,
38     "material": null
39 },
40 "raw_text": "1/7/25,7:40 AM DCMH.CO2039_Advanced Programming HO
    CHI MINH CITY ... (truncated)"
41 }

```

Field glossary

Table 1.1: Field glossary for syllabus.v1

Key	Type	Description
schema_version	string	Logical schema tag (e.g., syllabus.v1).
metadata	object	Document-level metadata used for auditing/filtering.
doc_type	string	Constant "syllabus".
course_id	string	Course code inferred from header/filename.
source_file	string	Absolute/relative path to the page image.
page_index	int	Page index of the syllabus image.
language	string	Language tag of OCR text.
ocr_engine	string	Engine/version used (e.g., PaddleOCR 3.2).
tractor_version	string	Version of the syllabus extractor.

Continued on next page

Table 1.1 (continued)

Key	Type	Description
timestamp	string	ISO-8601 time of extraction.
course_info	object	Canonical header fields parsed from the page.
title	string	Course title.
course_id	string	Redundant copy for convenience/joins.
credits	int	Local credit units (ECTS derivation optional).
applied_semester	string	Semester tag if detected (e.g., HK202).
assessments	array	Rows such as Lectures, Tutorial, Labs, Projects, etc.
name	string	Component name.
hours	intnull	Parsed cell values (robust to missing cells).
ratio	intnull	Weight (%) for the component.
evaluation_type	array	Optional sub-entries (e.g., Midterm/Final).
name	string	Sub-component name.
ratio	intnull	Weight (%).
duration_min	intnull	Duration in minutes if available.
course_des	object	Optional course description/material notes.
raw_text	string	Concatenated OCR text for provenance/debugging.

1.2.6 Notes on Robustness

- Spatial matching uses padding to tolerate OCR alignment noise; cells are chosen with inside or overlap as needed.
- Column/row anchors accept variants (e.g., “Hours” vs “Lessons”, “Duration” vs “Time”), and English-only exam terms are whitelisted.

1.3 Material Extraction

1.3.1 Goal

For each slide image, persist (i) the full OCR text and (ii) document meta-data needed downstream (course id, page index, engine, timestamp, etc.), emitting a course-level JSON bundle under schema material.v1. :contentReference[oaicite:0]index=0

1.3.2 Inputs

The extractor consumes per-slide OCR output (raw text) together with the source path so we can infer the chapter number from folder names like Chapter_7/slide_012.png.

1.3.3 Method

1. **Metadata assembly.** Build a Metadata record with doc_type="slide", course_id, source_file, page_index, language, ocr_engine, extractor_version, and timestamp.
2. **Slide object.** Create a Material object with fields course, schema_version="material.v1", metadata, and raw_text (the full OCR text for that slide).
3. **Chapter inference.** Derive chapter_num from source_file via the regex chapter[_-\\s]*(\\d+); default to 0 if absent.
4. **Serialization.** Write a course-level JSON: top-level course, course_id, schema_version, and slides[] where each slide carries page_index, chapter_num, source_file, metadata{...}, and raw_text.

1.3.4 Data Model & Metadata

The Python dataclass defines Material as: {course, schema_version="material.v1", metadata, raw_text}. A real example from your run is shown below.

```
{
  "course": "Advanced_Programming",
  "course_id": "CO2039",
  "schema_version": "material.v1",
  "slides": [
    {
      "page_index": 0,
      "chapter_num": 1,
      "source_file": "/.../Chapter_1/slide_001.png",
      "metadata": {
        "doc_type": "slide",
        "course_id": "CO2039",
        "source_file": "/.../Chapter_1/slide_001.png",
        "page_index": 0,
        "language": "en",
        "ocr_engine": "PaddleOCR 3.2",
        "extractor_version": "1.0.0",
        "timestamp": "2025-10-31T12:28:18+07:00"
      },
      "raw_text": "DAI HOC QUOC GIA BK TP.HCM ... Chapter 1: Object
        Oriented Programming ..."
    },
    { "page_index": 1, "chapter_num": 1, "source_file": "/.../slide_002.png",
      "metadata": { "...": "..." }, "raw_text": "CONTENT 01 INTRODUCTION
        ..." }
    // ... more slides ...
  ]
}
```

Notes.

- **Titles/items:** Helper functions to extract slide titles exist but are commented out in the current version; JSON omits an items array and keeps only `raw_text + metadata`.
- **Persistence:** The writer ensures a debug-friendly, indented JSON with UTF-8 and `ensure_ascii=false`.

1.4 Text Chunking

1.4.1 Scope and role

Text chunking is part of *preprocessing*. The end-to-end indexing pipeline then applies embedding and FAISS indexing on the produced chunks; in this section we focus solely on chunking.

(Pipeline composition is defined in `indexing_pipeline.py`: *chunking* $\rightarrow$ *embedding* $\rightarrow$ *indexing*.)

1.4.2 Inputs

- **Syllabus JSON** (`syllabus.v1`): we selectively chunk (i) course info, (ii) assessments, (iii) course description, and (iv) raw OCR text if present.
- **Material JSON** (`material.v1`): we chunk each slide's `raw_text` independently with slide metadata.

1.4.3 Method

We produce overlapping windows of **default 512 tokens** with **50-token** overlap. Token counts use `tiktoken` with the `cl100k_base` vocabulary when available; otherwise we fall back to a character-based approximation (1 token $\approx$ 4 chars).

- **Tokenizer**: try `tiktoken.get_encoding("cl100k_base")`; if unavailable, degrade to the 4-char heuristic.
- **Windowing**: slide a window of size S with overlap O over the token sequence; for the fallback, use $S \times 4$ chars and $O \times 4$ char overlap.
- **Empty/short inputs**: return no chunks if text is empty; if the whole text fits into one window, emit a single chunk with suffix `-0000`.

Interfaces (abridged).

```

1 chunk_text(text, metadata, chunk_id_prefix,
2           chunk_size=512, overlap=50, encoding=None) -> List[DocChunk]
3
4 chunk_syllabus(syllabus_json, chunk_size=512, overlap=50) -> List[DocChunk
5                  ]
6
7 chunk_material(material_json, chunk_size=512, overlap=50) -> List[
8                  DocChunk]

```

Each function attaches the provided/stemmed Metadata to every output chunk (doc type, course id, source file, page index, language, OCR engine, extractor version, timestamp).

1.4.4 Chunk IDs, prefixes, and metadata

- **Stable IDs:** <prefix>-NNNN with zero-padded indices (-0000, -0001, ...).
- **Material prefix:** <course_id>-chapter-<n>-slide-<p> (e.g., CO2039-chapter-1-slide-003).
- **Syllabus prefixes:**
 - <course_id>-syllabus-info
 - <course_id>-syllabus-assessments
 - <course_id>-syllabus-description
 - <course_id>-syllabus-raw
- **Metadata propagation:** all chunks inherit the original page- and file-level metadata for downstream filtering and attribution.

1.4.5 Complexity and expected chunk count

Let T be the token length of the input and (S, O) the window/overlap:

$$\text{chunks}(T; S, O) \approx \left\lceil \frac{\max(T - S, 0)}{S - O} \right\rceil + 1.$$

- T : the total number of tokens in the input document.
- S : the window size (in tokens), i.e., the maximum number of tokens allowed in a single chunk.
- O : the overlap size (in tokens) shared between two consecutive chunks.
- $S - O$: the stride, representing how many new tokens each window advances.

The implementation encodes once per document when using tiktoken to avoid repeated tokenization.

Example. Consider the following short sentence (given $T = 20$ tokens, here we treat each word as one token):

Modern computer networks connect clients and servers through routers and switches to deliver reliable web applications for users worldwide today.

Let the window size be $S = 8$ tokens and the overlap $O = 6$ tokens. The stride (step) between two consecutive windows is

$$\text{stride} = S - O = 8 - 6 = 2.$$

We have $T = 20$, so

$$T - S = 20 - 8 = 12, \quad \frac{T - S}{S - O} = \frac{12}{2} = 6,$$

and therefore

$$\text{chunks}(20; 8, 6) = \left\lceil \frac{20 - 8}{8 - 6} \right\rceil + 1 = \lceil 6 \rceil + 1 = 7.$$

Concretely, the seven overlapping chunks (in terms of token indices) are:

- Chunk 1: tokens [1, 8]
- Chunk 2: tokens [3, 10]
- Chunk 3: tokens [5, 12]
- Chunk 4: tokens [7, 14]
- Chunk 5: tokens [9, 16]
- Chunk 6: tokens [11, 18]
- Chunk 7: tokens [13, 20]

Here the stride is small (2 tokens), so adjacent chunks have a large overlap ($O = 6$ tokens), which improves recall at the cost of more chunks per document.

1.4.6 Example outputs

```
1 {  
2   "id": "CO2039-chapter-1-slide-003-0002",  
3   "text": "...windowed plaintext from slide 3...",  
4   "metadata": {  
5     "doc_type": "slide", "course_id": "CO2039",  
6     "source_file": ".../Chapter_1/slide_003.png",  
7     "page_index": 3, "language": "en",  
8     "ocr_engine": "PaddleOCR 3.2",  
9     "extractor_version": "1.0.0",  
10    "timestamp": "2025-10-31T12:28:18+07:00"  
11  }  
12 }
```

Chapter 2

Vector Embedding

Vector embeddings convert textual content into fixed-dimensional numerical vectors that preserve semantic similarity. In the context of our RAG system, embeddings form the core representation used to search and retrieve relevant syllabus and lecture material.

2.1 Concept

A *vector embedding* is a dense, continuous-valued numerical representation of text. Given a string—ranging from a single sentence to a multi-paragraph chunk—the embedding model maps it to a vector:

$$f : \text{text} \rightarrow \mathbb{R}^d$$

Two key principles define embedding usefulness:

- **Semantic proximity:** semantically similar text segments lie near each other in vector space.
- **Task transferability:** the model generalizes across tasks such as search, question answering, and classification.

Cosine similarity is used as the distance metric:

$$\text{cosine}(u, v) = \frac{u \cdot v}{\|u\| \|v\|}$$

Because vectors produced by our embedding model are L2-normalized, cosine similarity becomes equivalent to inner-product similarity. This allows us to use the FAISS IndexFlatIP index for efficient retrieval.

2.2 Embedding Model

To generate embeddings, the system uses the **sentence-transformers/all-MiniLM-L6-v2** model.

This model produces compact 384-dimensional embeddings. semantic quality and computational efficiency.

Embedding generation in the indexing pipeline is done in batches using:

```

1  chunk_texts = [chunk.text for chunk in chunks]
2  chunk_ids = [chunk.id for chunk in chunks]
3
4  all_embeddings = []
5  for i in tqdm(range(0, len(chunk_texts), batch_size), desc="Embedding"):
6      batch = chunk_texts[i:i + batch_size]
7      batch_embeddings = embedding_model.embed_batch(batch)
8      if isinstance(batch_embeddings, list):
9          all_embeddings.extend(batch_embeddings)
10     else:
11         all_embeddings.extend(batch_embeddings.tolist())

```

(See embedding integration in the indexing pipeline: [\[CITE: indexing_pipeline.py\]](#).)

Example.

Input: *“PPL is the hardest course in HCMUT!”*

Output (384-dimensional vector, truncated):

$$\mathbf{e} = \begin{bmatrix} -6.67869747 \times 10^{-2} & 1.13732656 \times 10^{-2} & 5.77662513 \times 10^{-3} \\ -2.20818091 \times 10^{-2} & -1.42803207 \times 10^{-1} & -4.07201834 \times 10^{-2} \\ 4.65728492 \times 10^{-2} & -8.58380832 \times 10^{-3} & -5.98169118 \times 10^{-2} \\ -2.43831854 \times 10^{-2} & -2.83657387 \times 10^{-2} & -2.22538076 \times 10^{-2} \\ 3.11421230 \times 10^{-2} & -1.96713191 \times 10^{-2} & -2.50504017 \times 10^{-2} \\ 8.42615683 \times 10^{-3} & -3.56746353 \times 10^{-2} & 1.24848075 \times 10^{-2} \\ 4.07706238 \times 10^{-2} & -1.14178978 \times 10^{-1} & \dots \end{bmatrix}$$

Explain:

- The vector is a 384-dimensional sentence embedding produced by all-MiniLM-L6-v2.
- Component values lie roughly within $\approx [-0.2, 0.09]$, which is typical due to normalization and mean-pooling.
- No NaN or infinite values are present; small magnitudes (10^{-3} – 10^{-2}) are normal numerical fluctuations.
- Individual dimensions are not interpretable; the embedding is meaningful only as a whole.
- The vector is valid and suitable for cosine or inner-product search in the FAISS index.

2.3 Relationship with Chunking

Embeddings are generated **after** the input text is segmented into overlapping chunks. The chunking process is defined formally in Section ??, where the expected number of chunks is derived based on the window size S , overlap O , and document length T .

In practice, chunking is implemented in `chunk_text` and related functions ([CITE: [chunking.py](#)]), which:

1. tokenize the text using `tiktoken`,
2. apply a sliding window with stride $S - O$,
3. and produce `DocChunk` objects enriched with metadata.

```
1 class Embedding():
2     def __init__(self):
3         self.model = load_model("embed")
4
5     def embed(self, text: str) -> list[float]:
6         return self.model.encode(text).tolist()
7
8     def embed_batch(self, texts: list[str]) -> list[list[float]]:
9         return self.model.encode(texts).tolist()
10
11     def save_index(self, index: IndexFlatIP, path: Path) -> None:
12         index.save(str(path))
13
14     def load_index(self, path: Path) -> IndexFlatIP:
15         return IndexFlatIP.load(str(path))
```

2.4 Embedding Generation in the Pipeline

Embedding generation in the pipeline ([CITE: [indexing_pipeline.py](#)]) is performed in mini-batches:

```
1 for i in range(0, len(chunk_texts), batch_size):
2     batch = chunk_texts[i : i + batch_size]
3     batch_embeddings = embedding_model.embed_batch(batch)
```

Key steps

1. Load chunks from syllabus and material documents.
2. Generate embeddings in memory-efficient batches.
3. Store embeddings as a NumPy float32 matrix.
4. Insert vectors into the FAISS index.

This design ensures:

- No repeated tokenization (documents encoded once),
- Fast batch inference,
- Metadata preserved for each embedding.

2.5 Embedding Indexing: FAISS IndexFlatIP

We use FAISS IndexFlatIP (Inner Product index) because:

- Embeddings are unit-normalized $\rightarrow$ cosine similarity = inner product,
- Fast dense vector search,
- CPU-optimized, simple, and reliable.

Index structure:

- `index.faiss` — raw vector matrix,
- `metadata.json` — mapping between vector ID and chunk metadata.

The FAISS index is built through:

```
1 index = create_index(embedding_dim=384)
2 add_documents(index, embeddings_array, chunk_ids)
3 save_index(index, index_dir, metadata_map)
```

2.6 Summary

Vector embeddings form the semantic backbone of the RAG retrieval system:

- Encode meaning in dense numerical form,
- Enable fast similarity search using FAISS,
- Integrate seamlessly with sliding-window chunking,
- Enhance retrieval quality for syllabus and lecture materials.

The entire pipeline—from text extraction, to chunking, embedding, and indexing—is modular, scalable, and optimized for high-quality retrieval across thousands of document chunks.

Chapter 3

Indexing

Indexing is the final stage of the retrieval-side pipeline. After semantic embeddings are generated from syllabus and slide text (Chapter 2), the system organizes these vectors into a Vectordatabase whose structure supports fast, scalable, and accurate top- k similarity search.

3.1 Overview

The goal of indexing is to transform thousands of chunk embeddings into:

- a **vector index** that supports fast cosine-similarity search,
- a **metadata map** that links each embedding to its original syllabus/slide,
- a **reusable index store** that can be loaded instantly during inference.

We adopt **FAISS IndexFlatIP**, leveraging inner-product similarity on normalized vectors. This provides exact (non-approximate), deterministic retrieval while remaining fast on CPU.

3.2 Vector Indexing

The complete indexing workflow is implemented in `preprocessing/indexing_pipeline.py`.

It consists of the following steps:

1. Load syllabus and material JSON files.
2. Chunk all documents into token windows.
3. Generate embeddings in batches.
4. Normalize vectors and add them to FAISS.
5. Write index + metadata + debug artifacts.

3.2.1 Chunk Collection

All chunks are generated via:

from `.chunking` import `chunk_syllabus`, `chunk_material`

The relevant code:

```
1 def process_course(data_root, course_name, chunk_size=512, overlap=50):
2     all_chunks = []
3     syllabus_files = load_syllabus_files(data_root, course_name)
4     for syl_json in syllabus_files:
5         chunks = chunk_syllabus(syl_json, chunk_size, overlap)
6         all_chunks.extend(chunks)
7
8     material_json = load_material_file(data_root, course_name)
9     if material_json:
10         chunks = chunk_material(material_json, chunk_size, overlap)
11         all_chunks.extend(chunks)
12
13     return all_chunks
```

Each chunk is a `DocChunk` containing text, ID, and metadata.

3.2.2 Creating the FAISS Index

FAISS index creation uses IndexFlatIP (inner-product index) from `models/indexing.py`.

```
1 def create_index(embedding_dim: int):  
2     return IndexFlatIP(embedding_dim)
```

This index performs **exact cosine similarity** search when vectors are normalized.

3.2.3 Adding Embeddings to Index

Before storing vectors, embeddings are normalized:

```
1 def add_documents(index, embeddings, ids):  
2     norms = np.linalg.norm(embeddings, axis=1, keepdims=True)  
3     norms[norms == 0] = 1  
4     normalized = embeddings / norms  
5     index.add(normalized.astype('float32'))
```

This ensures:

$$\text{cosine}(u, v) = u \cdot v$$

FAISS then stores all vectors in contiguous memory for SIMD-optimized search.

3.2.4 Metadata Store

Metadata is generated via:

```
1 metadata_map = chunks_to_metadata_map(chunks)
```

Each vector has a corresponding entry:

```
{  
    "id": "CO1005-slide-003-0004",
```

```

"text": "... chunk text ...",
"metadata": {
    "doc_type": "slide",
    "course_id": "CO1005",
    "source_file": "...",
    "page_index": 3,
    "language": "en",
    "ocr_engine": "PaddleOCR 3.2"
}
}

```

This enables full traceability during retrieval.

3.2.5 Saving the Index

The index and metadata are written to disk:

```

1 def save_index(index, path, metadata_map):
2     index_path = path / "index.faiss"
3     faiss.write_index(index, str(index_path))
4
5     metadata_path = path / "metadata.json"
6     with open(metadata_path, "w") as f:
7         json.dump(metadata_map, f, indent=2)

```

(Source: models/indexing.py)

Debug visualizations (PCA plots, similarity heatmaps) are also created automatically by:

```
debug_index_during_build(...)
```


3.3 Searching

During inference, search is performed by:

```
def search(index, query_embedding, k=5):
    query_norm = np.linalg.norm(query_embedding)
    normalized_q = (query_embedding / query_norm).astype('float32')
    distances, indices = index.search(normalized_q.reshape(1, -1), k)
    return distances[0], indices[0]
```

Steps:

1. Embed query into 384-d unit vector.
2. Perform FAISS top- k search.
3. Convert FAISS row IDs $\rightarrow$ chunk IDs.
4. Lookup metadata for context reconstruction.

Latency is typically **1–10 ms** for thousands of chunks.

3.4 End-to-End Indexing Command

The full indexing command implemented in run.py:

```
python run.py --index \  
  --chunk-size 512 \  
  --chunk-overlap 50 \  
  --batch-size 32 \  
  --only-course CO1005_Introduction_to_Computing
```

Internally:

```
if args.index:  
    pipeline_index(  
        data_root=data_root,
```

```
index_dir=index_dir,  
chunk_size=args.chunk_size,  
overlap=args.chunk_overlap,  
batch_size=args.batch_size,  
only_course=args.only_course  
)
```

3.5 Summary

Indexing transforms a corpus of chunk embeddings into a fast and interpretable retrieval structure. FAISS IndexFlatIP provides exact cosine similarity search while maintaining CPU efficiency, and the metadata store enables precise linking of retrieved vectors back to syllabus and slide content. This chapter completes the retrieval preparation pipeline, enabling the RAG system described in later chapters.

Chapter 4

Add-ons

Source code repository:  https://github.com/HatakekkSheeshh/CSE_course_RAG.git